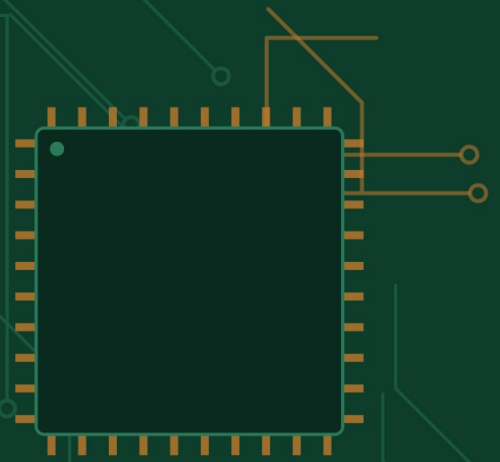
PERTEMUAN 11

Merancang Firmware Rapi: FSM dan Debugging



Dari kode spageti ke mesin keadaan, lalu membaca pesan crash

Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

Tujuan Pembelajaran

- Menjelaskan mengapa kode **spageti** sulit dirawat
- Merancang **FSM**: keadaan, peristiwa, transisi, aksi
- Menulis FSM *non-blocking* dan memecah kode ke **.h/.cpp**
- Men-*debug* dengan log, `assert()`, membagi dua, *rubber duck*
- Membaca pesan **Guru Meditation Error** dan *backtrace*

Peta Materi Hari Ini

01 Kode spageti vs FSM

02 Keadaan, peristiwa, transisi, aksi

03 FSM di C++: enum class + switch

04 Modul .h dan .cpp

05 Debugging sistematis

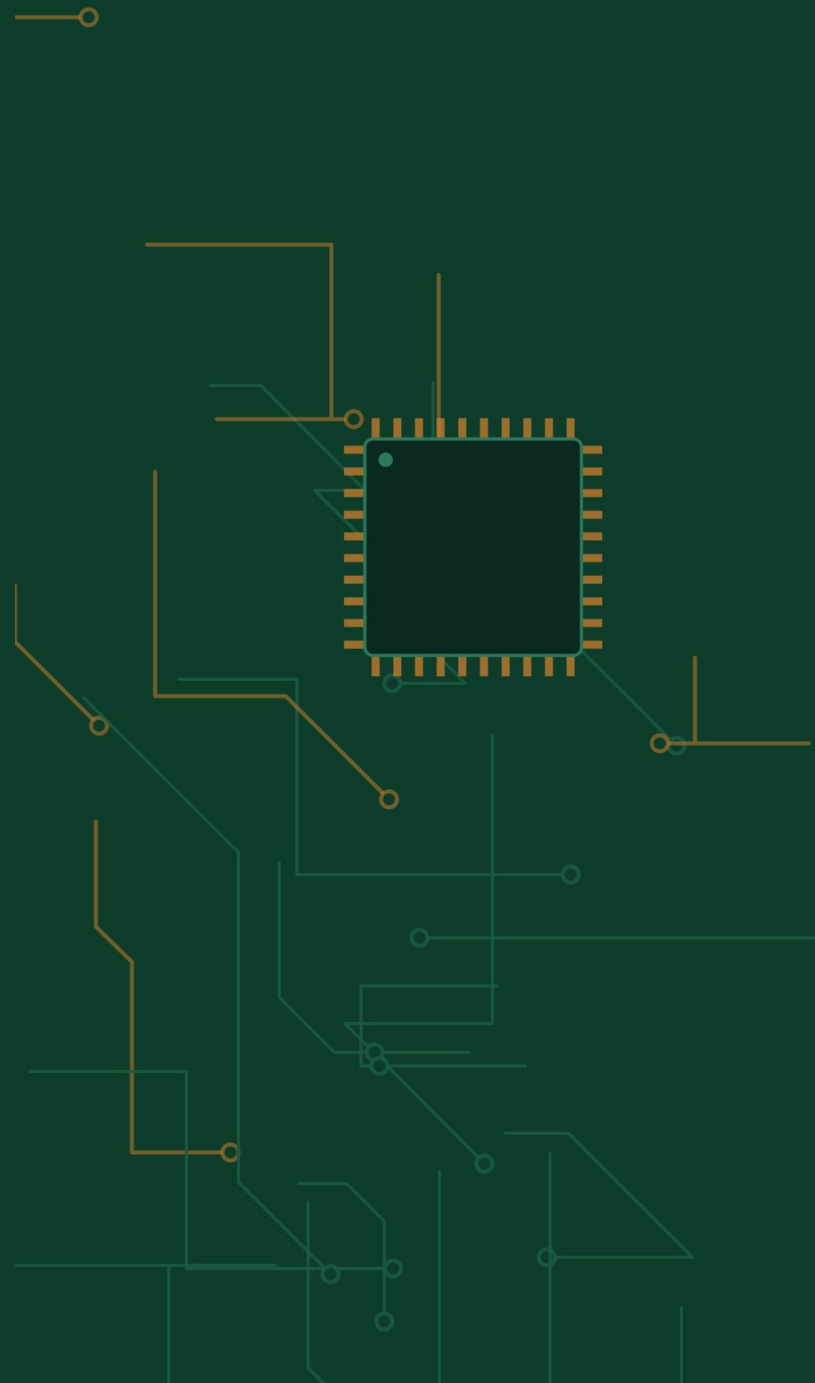
06 Membaca pesan crash

07 Praktikum 1–3

01

Dari Spageti ke FSM

Menata logika menjadi keadaan dan transisi



Kode Spageti vs Mesin Keadaan

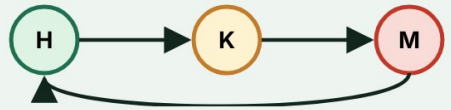
Kode "spageti"




```
void loop() {
  if (digitalRead(4) == LOW) minta = true;
  if (fase == 0 && !sudahKuning) {
    digitalWrite(27, HIGH);
    delay(8000);        // tombol tak terbaca!
    if (minta && !sudahMerah) fase = 2;
    else { fase = 1; sudahKuning = true; }
  } else if (fase == 1 || (minta && fase != 2)) {
    digitalWrite(27, LOW);
    digitalWrite(26, HIGH);
    delay(2000); fase = 2; minta = false;
  } else if (fase == 2 && sudahKuning) { /* ... */ }
}
```

- `delay()` membekukan program: tombol tidak terbaca selama 8 detik.
- Bendera bertebaran: 3 bendera = 2^3 = 8 kombinasi yang harus dipikirkan.
- Fitur baru berarti membongkar semua `if`.

Mesin keadaan (FSM)




```
switch (keadaan) {
  case Keadaan::HIJAU:
    if (lama >= LAMA_HIJAU_MS ||
        (adaPenyeberang && lama >= LAMA_HIJAU_MIN_MS))
      masukKeadaan(Keadaan::KUNING, "...");
    break;
  case Keadaan::KUNING:
    if (lama >= LAMA_KUNING_MS)
      masukKeadaan(Keadaan::MERAH, "...");
    break;
  case Keadaan::MERAH:  /* pola sama */
    break;
}
```

- Selalu tepat **satu** keadaan aktif, mudah ditebak.
- Tanpa `delay()`: tombol diperiksa ribuan kali per detik.
- Fitur baru = tambah satu keadaan (satu `case`).

Ikon: Font Awesome Free (CC BY 4.0)

Anatomi FSM: Gerbang Tol

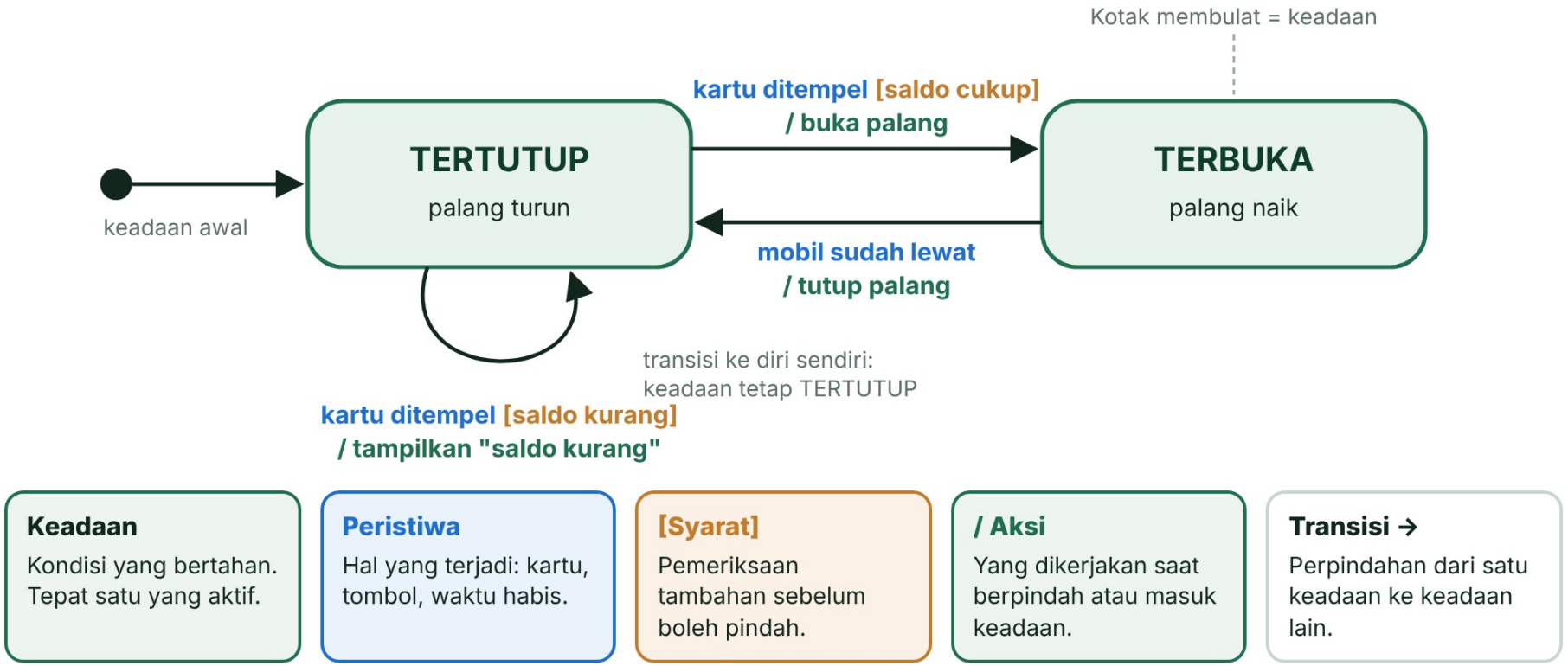
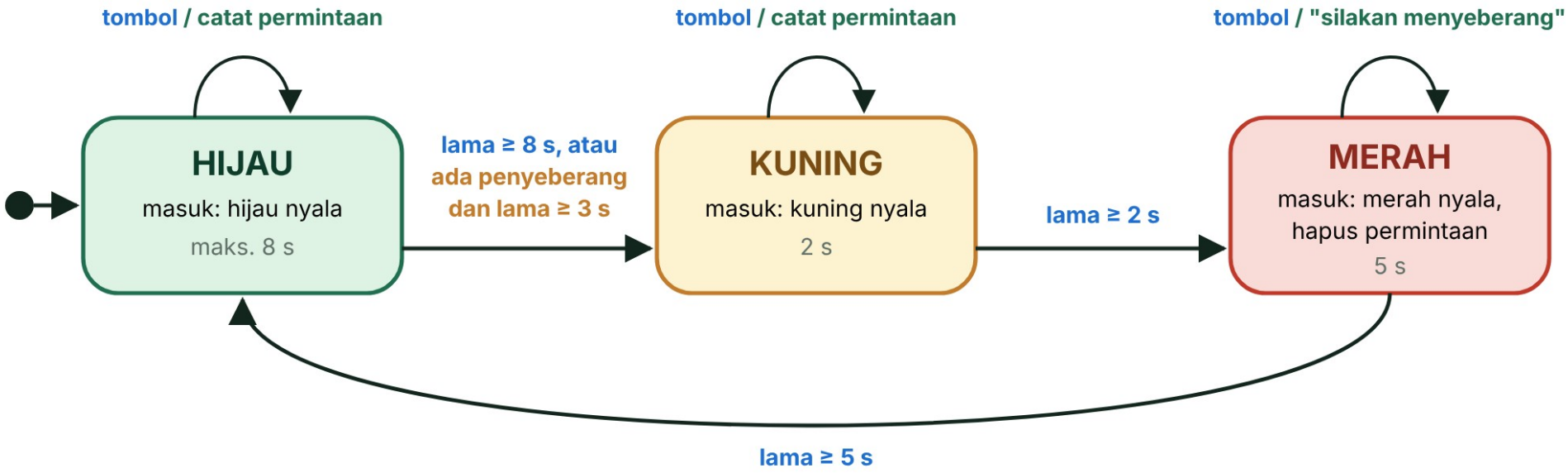


Diagram Keadaan Lampu Lalu Lintas



Siklus normal tanpa penyeberang (15 s):



Tabel Transisi = Daftar Uji

Keadaan	Peristiwa [syarat]	Aksi	Berikutnya
HIJAU	lama ≥ 8 s	kuning nyala	KUNING
HIJAU	[ada penyeberang] dan lama ≥ 3 s	kuning nyala	KUNING
HIJAU/KUNING	tombol ditekan	catat permintaan	(tetap)
KUNING	lama ≥ 2 s	merah nyala, hapus permintaan	MERAH
MERAH	lama ≥ 5 s	hijau nyala	HIJAU
MERAH	tombol ditekan	"silakan menyeberang"	(tetap)

Pola FSM: enum class + switch

src/main.cpp

```
enum class Keadaan { HIJAU, KUNING, MERAH };
Keadaan keadaan = Keadaan::HIJAU;
unsigned long waktuMasuk = 0;

void loop() {
  unsigned long lama = millis() - waktuMasuk;
  if (tombolDitekan()) adaPenyeberang = true;
  switch (keadaan) {
    case Keadaan::HIJAU:
      if (lama >= LAMA_HIJAU_MS)
        masukKeadaan(Keadaan::KUNING, "habis");
      break;
    // KUNING, MERAH: pola sama
  }
}
```

- `enum class`: daftar keadaan bernama
- `lama` = waktu sejak masuk keadaan
- `switch`: satu `case` per keadaan
- Tanpa `delay()`: tombol selalu terbaca

Fungsi Masuk-Keadaan

src/main.cpp

```
void masukKeadaan(Keadaan baru, const char *alasan) {
  keadaan = baru;
  waktuMasuk = millis();       // "lama" mulai dari 0
  switch (baru) {
    case Keadaan::MERAH:
      lampuTampilkan(Warna::MERAH);
      adaPenyeberang = false;  // permintaan terlayani
      break;
    // ... HIJAU, KUNING
  }
  Serial.printf("KEADAAN -> %s (%s)\n",
                namaKeadaan(baru), alasan);
}
```

- Jangan `delay()` di dalam `case`
- Ubah `keadaan` **hanya** lewat fungsi ini
- Akhiri setiap `case` dengan `break`
- Log otomatis setiap pindah keadaan

Fitur Baru = Keadaan Baru

1

Tambah nama

`KEDIP_MALAM` di `enum class Keadaan`.

2

Tambah case

Kuning berkedip tiap 500 ms tanpa `delay()`.

3

Aksi masuk

Tambah satu `case` di `masukKeadaan()`.

4

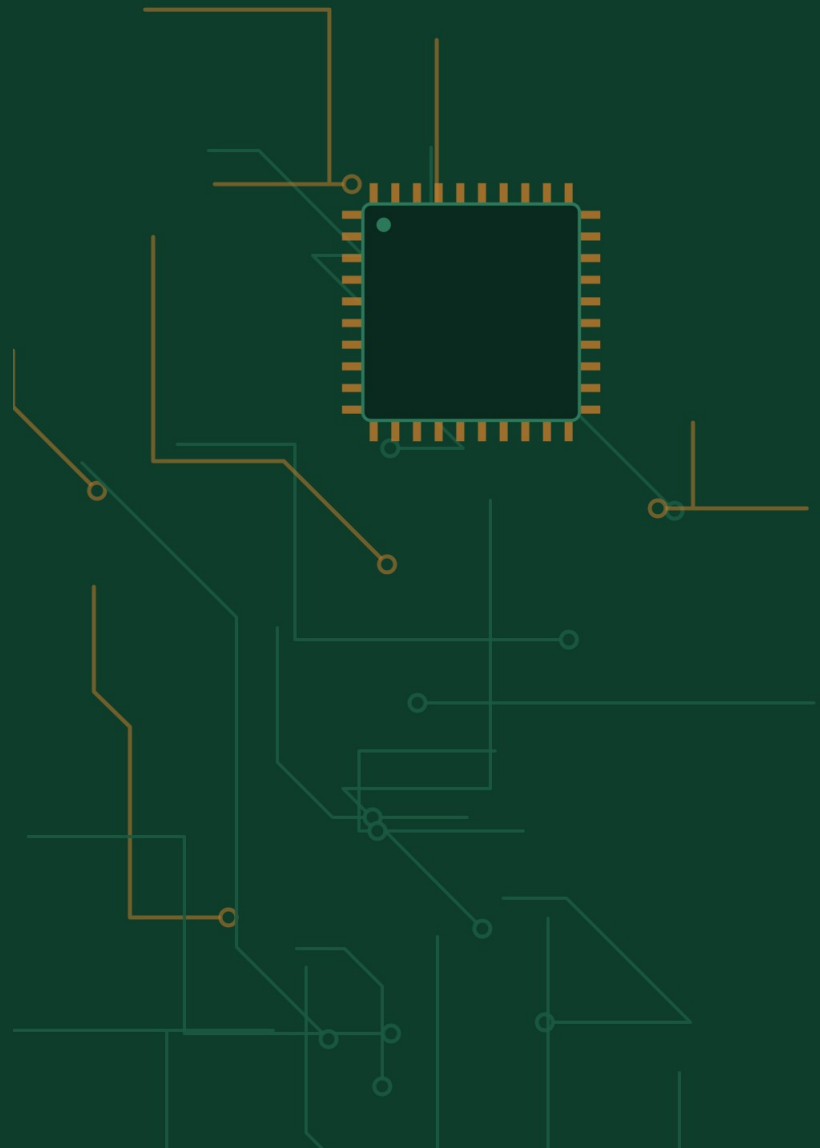
Nama log

Tambah satu `case` di `namaKeadaan()`.

02

Kode Rapi dan Debugging

Modul .h/.cpp, log, assert, membagi dua



Memecah Kode menjadi Modul

PlatformIO (VS Code)

```
01-lalu-lintas-fsm/
├─ platformio.ini
├─ diagram.json
├─ include/
│  ├─ konfigurasi.h   pin & durasi
│  ├─ lampu.h         menu lampu
│  └─ tombol.h        menu tombol
└─ src/
   ├─ main.cpp        FSM
   ├─ lampu.cpp       isi lampu
   └─ tombol.cpp      isi tombol
```

salin isi tiap berkas

Wokwi (peramban)

`sketch.ino` `konfigurasi.h` `lampu.h` `lampu.cpp` `tombol.h` `tombol.cpp` `diagram.json`

- Semua berkas **sejajar**, tanpa folder.
- Isi `sketch.ino` = isi `src/main.cpp`.
- `#include "lampu.h"` tetap sama.

.h = menu (antarmuka)

Daftar fungsi yang boleh dipanggil modul lain. Diawali `#pragma once`.

```
void lampuTampilkan(Warna w);
```

.cpp = dapur (isi)

Cara kerja fungsinya. Variabel `static` = rahasia modul.

```
digitalWrite(PIN_MERAH, ...);
```

main.cpp = pengatur

Berisi FSM: memutuskan kapan pindah keadaan, lalu memanggil modul.

```
#include "lampu.h"
```

Ikon: Font Awesome Free (CC BY 4.0)

.h = Menu, .cpp = Dapur



.h (berkas kepala)

- Deklarasi fungsi dan tipe
- Diawali `#pragma once`
- Disimpan di `include/`
- `void lampuTampilkan(Warna w);`



.cpp (berkas sumber)

- Isi fungsi
- Variabel `static` = rahasia modul
- Disimpan di `src/`
- `digitalWrite(PIN_MERAH, ...);`

Alur Debugging yang Sistematis

1 — Amati gejala
Apa yang salah? Kapan? Seberapa sering?
`Serial Monitor`

2 — Ulangi (reproduksi)
Temukan langkah pasti yang selalu memunculkan bug.
`catat langkah`

3 — Kumpulkan bukti
Log, nilai variabel, pesan crash.
`log_i()`
`backtrace`

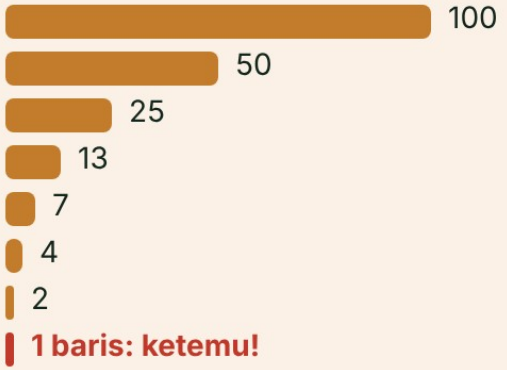
4 — Buat dugaan
Satu penyebab yang bisa diuji.
`rubber duck`

5 — Uji SATU perubahan
Dugaan salah? Kembali ke langkah 4.
`bagi dua`
`assert()`

6 — Perbaiki & uji ulang
Bug hilang? Fitur lain masih berjalan?
`uji ulang`

Teknik membagi dua

Bug ada di antara 100 baris? Matikan/cek separuhnya, lalu separuh lagi:



100 baris → **7 langkah**.
1.000 baris → cukup **10 langkah**.

Makro Log dan CORE_DEBUG_LEVEL

Level	Nama	Makro	Contoh pemakaian
0	NONE	—	Semua log mati (produk jadi)
1	ERROR	log_e()	Sensor tidak menjawab
2	WARN	log_w()	Nilai janggal, belum fatal
3	INFO	log_i()	Program mulai, koneksi berhasil
4	DEBUG	log_d()	Detail untuk men-debug
5	VERBOSE	log_v()	Sangat rinci

Empat Alat Debugging



Serial terstruktur

Tag + waktu + nilai; cetak saat berubah saja.



assert()

Jaring pengaman untuk kondisi yang mustahil.



Membagi dua

Buang separuh kemungkinan setiap langkah.



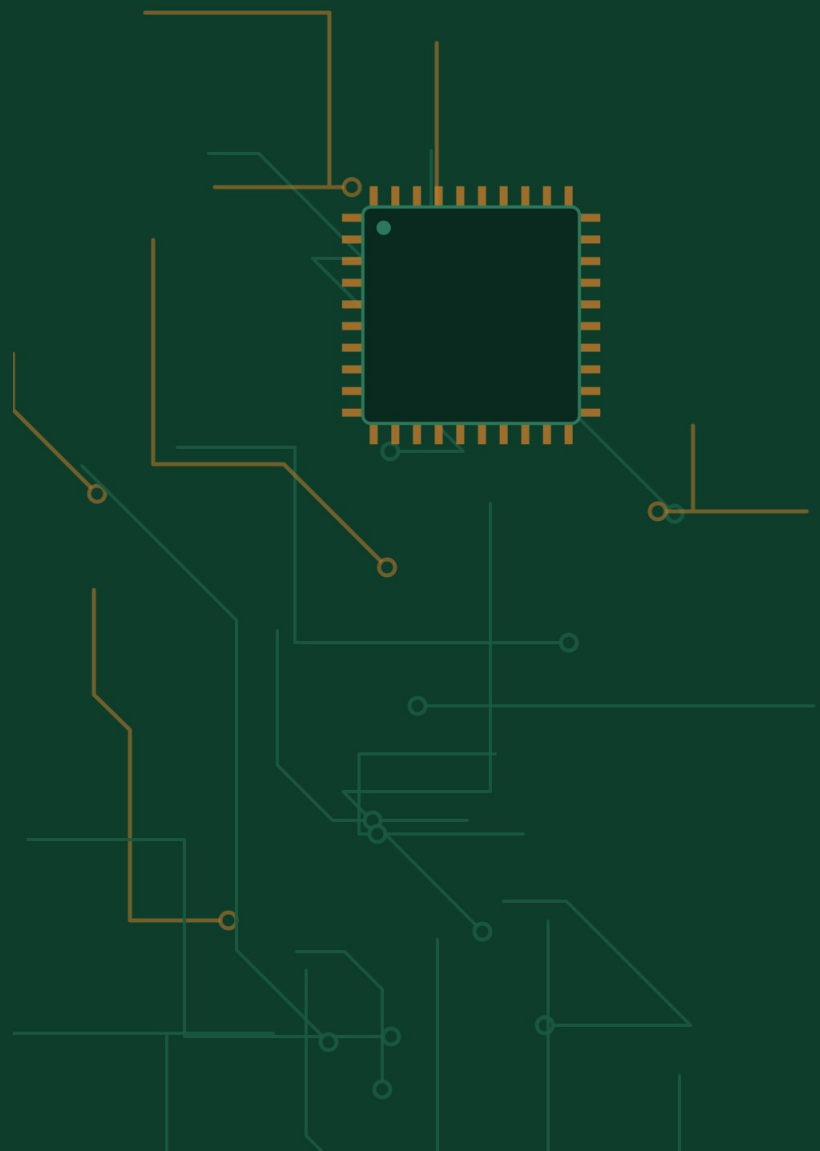
Rubber duck

Jelaskan kode baris demi baris.

03

Membaca Pesan Crash

Guru Meditation Error, register, dan backtrace



Anatomi Guru Meditation Error

```
[INFO] Perintah: baca
Guru Meditation Error: Core  1 panic'ed (LoadProhibited). (1) (2)
Exception was unhandled.
Core  1 register dump:
PC      : 0x400d1907  PS      : 0x00060230  A0  : 0x800d1a00 … (3)
…  (register A1–A15 dipotong)
SAR     : 0x0000000a  EXCCAUSE: 0x0000001c (4)
EXCVADDR: 0x00000004  LBEG    : 0x4008516d  … (5)

Backtrace: 0x400d1904:0x3ffb20b0 0x400d19fd:0x3ffb20d0 (6)
           0x400d1be1:0x3ffb20f0 0x400d4fa4:0x3ffb2130 …
  #0  0x400d1904 in crashBacaNull() at src/main.cpp:114 (7)
  #1  0x400d19fd in jalankanPerintah(...) at src/main.cpp:87
  #2  0x400d1be1 in loop() at src/main.cpp:65

ELF file SHA256: 331765eca
Rebooting... (8)
rst:0xc (SW_CPU_RESET),boot:0x12 (SPI_FAST_FLASH_BOOT)
[INFO] Penyebab reset terakhir: PANIC (crash/exception)
```

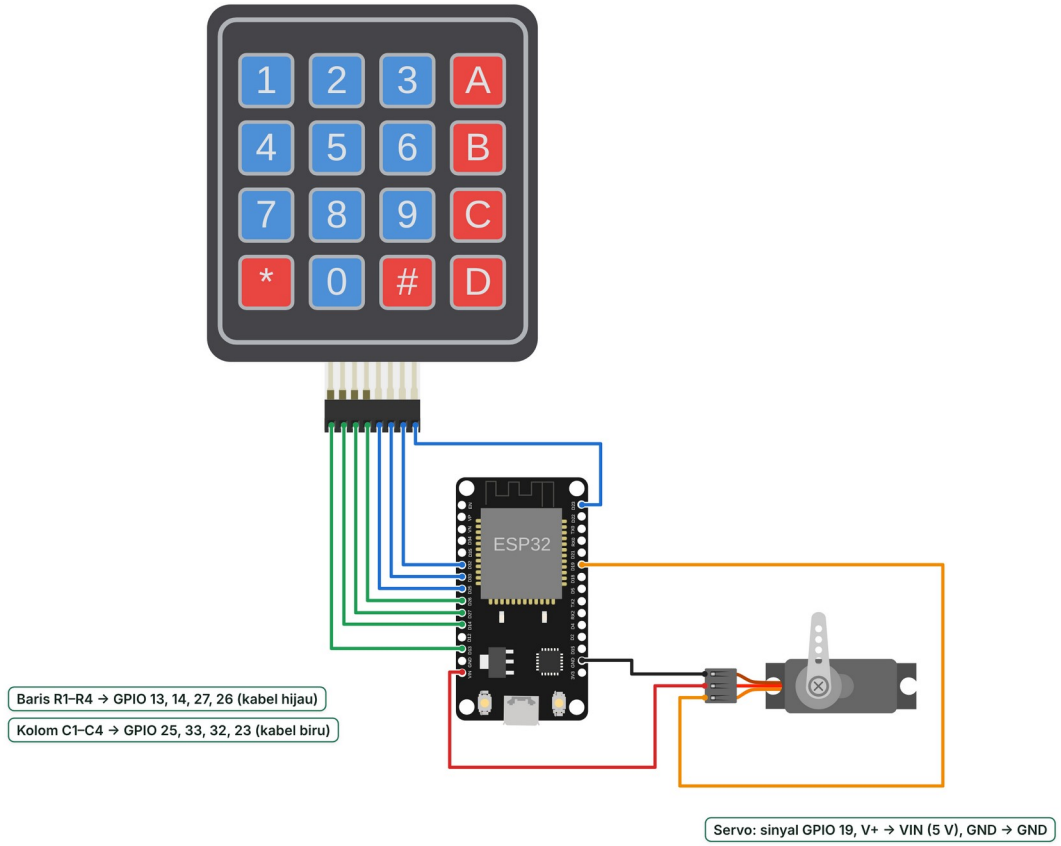
1. **Jenis galat.** LoadProhibited = CPU membaca alamat memori terlarang.
2. **Core 1** = inti tempat `loop()` berjalan.
3. **PC** (*program counter*): alamat instruksi saat crash.
4. **EXCCAUSE**: kode penyebab, 0x1c (28) = LoadProhibited.
5. **EXCVADDR**: alamat yang diakses. 0x00000004 = pointer null + offset 4.
6. **Backtrace**: jejak pemanggilan, fungsi terbaru paling kiri.
7. **Baris biru** ditambahkan filter `esp32_exception_decoder` di PlatformIO: fungsi + berkas:baris.
8. **Rebooting**: ESP32 di-reset otomatis, lalu program bisa membaca penyebab reset.

Pesan Crash yang Sering Muncul

Pesan	Arti	Penyebab umum
LoadProhibited / StoreProhibited	Akses alamat terlarang	Pointer null atau rusak
IntegerDivideByZero	Bagi bilangan bulat dengan nol	Pembagi bernilai 0
Stack canary watchpoint	Stack task meluap	Rekursi, larik lokal besar
Task watchdog got triggered	Task tidak memberi makan watchdog 5 s	Perulangan tanpa jeda
Brownout detector was triggered	Tegangan catu turun	Servo/Wi-Fi, catu lemah
assert failed	assert() gagal	Bug logika tertangkap

Praktikum 2: Pin Keypad yang Aman

- Baris R1–R4 → GPIO 13, 14, 27, 26
- Kolom C1–C4 → GPIO 25, 33, 32, 23
- Baris `INPUT_PULLUP`, kolom `OUTPUT` → bukan GPIO 34–39
- Hindari *strapping* 0, 2, 5, 12, 15
- Servo GPIO 19, V+ ke VIN (5 V)



Gambar komponen: @wokwi/elements (MIT)

Praktikum 3: Hasil Tiap Perintah

Perintah	Pesan (QEMU)	Penyebab menurut decoder
bagi0	IntegerDivideByZero	crashBagiNol(), baris 103
baca	LoadProhibited, EXCVADDR 0x00000004	crashBacaNull(), baris 114
assert	assert failed … main.cpp:140	cekSuhu(int), baris 140
macet	Task watchdog got triggered	crashMacet(), baris 132
rekursi	sisa stack 6172 → 892 byte	±176 byte per panggilan
null	QEMU diam; board: StoreProhibited	crashTulisNull()

Kuis Cepat



Crash LoadProhibited dengan EXCVADDR 0x00000004 paling mungkin berarti ...

A Pembagian dengan nol

B Membaca anggota struct lewat pointer null

C Tegangan catu turun

D Perulangan tanpa jeda

Rangkuman

- FSM: tepat satu **keadaan** aktif; berpindah karena **peristiwa** dan **syarat**
- FSM di C++: `enum class` untuk daftar keadaan, `switch` untuk aturannya, `millis()` untuk waktu
- Kode rapi: **.h** (menu) + **.cpp** (dapur), konstanta bernama, komentar "mengapa"
- *Debugging*: pesan log bertingkat (`log_e` … `log_d`), `assert()`, dan teknik membagi dua
- Crash: baca jenis galat, **PC**, **EXCVADDR**, lalu *backtrace* hasil decoder

Sebelum Pertemuan 12



Tugas

- Latihan 1: mode malam
- Latihan 5: detektif bug



Persiapan

- Baca modul Pertemuan 12: FreeRTOS
- Ingat: loop() berjalan sebagai task



Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas